

In [2]:

```
#basics
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import numpy as np

data=fetch_california_housing(as_frame=True)
df=pd.concat([data.data,data.target.rename('MedHouseVal')],axis=1)
df.head()
```

Out[2]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526162
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.663362
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.421975
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	2.728012
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	2.161934

In [3]:

```
data
```

Out[3]:

```
{'data':      MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude \
0      8.3252      41.0  6.984127   1.023810      322.0  2.555556      37.88
1      8.3014      21.0  6.238137   0.971880     2401.0  2.109842      37.86
2      7.2574      52.0  8.288136   1.073446      496.0  2.802260      37.85
3      5.6431      52.0  5.817352   1.073059      558.0  2.547945      37.85
4      3.8462      52.0  6.281853   1.081081      565.0  2.181467      37.85
...      ...      ...      ...      ...      ...      ...
20635   1.5603      25.0  5.045455   1.133333      845.0  2.560606      39.48
20636   2.5568      18.0  6.114035   1.315789      356.0  3.122807      39.49
20637   1.7000      17.0  5.205543   1.120092     1007.0  2.325635      39.43
20638   1.8672      18.0  5.329513   1.171920      741.0  2.123209      39.43
20639   2.3886      16.0  5.254717   1.162264     1387.0  2.616981      39.37
```

```
Longitude
0      -122.23
1      -122.22
2      -122.24
3      -122.25
4      -122.25
...      ...
20635   -121.09
20636   -121.21
20637   -121.22
20638   -121.32
20639   -121.24
```

[20640 rows x 8 columns],

'target': 0 4.526

```
1      3.585
2      3.521
3      3.413
4      3.422
...
20635   0.781
20636   0.771
20637   0.923
20638   0.847
20639   0.894
```

Name: MedHouseVal, Length: 20640, dtype: float64,

```
'frame':      MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude \
0      8.3252      41.0  6.984127   1.023810      322.0  2.555556      37.88
1      8.3014      21.0  6.238137   0.971880     2401.0  2.109842      37.86
2      7.2574      52.0  8.288136   1.073446      496.0  2.802260      37.85
3      5.6431      52.0  5.817352   1.073059      558.0  2.547945      37.85
4      3.8462      52.0  6.281853   1.081081      565.0  2.181467      37.85
...      ...      ...      ...      ...      ...      ...
20635   1.5603      25.0  5.045455   1.133333      845.0  2.560606      39.48
20636   2.5568      18.0  6.114035   1.315789      356.0  3.122807      39.49
20637   1.7000      17.0  5.205543   1.120092     1007.0  2.325635      39.43
20638   1.8672      18.0  5.329513   1.171920      741.0  2.123209      39.43
20639   2.3886      16.0  5.254717   1.162264     1387.0  2.616981      39.37
```

```
Longitude  MedHouseVal
0      -122.23      4.526
1      -122.22      3.585
2      -122.24      3.521
3      -122.25      3.413
4      -122.25      3.422
...      ...      ...
20635   -121.09      0.781
```

20636	-121.21	0.771
20637	-121.22	0.923
20638	-121.32	0.847
20639	-121.24	0.894

```
[20640 rows x 9 columns],
'target_names': ['MedHouseVal'],
'feature_names': ['MedInc',
'HouseAge',
'AveRooms',
'AveBedrms',
'Population',
'AveOccup',
'Latitude',
'Longitude'],
'DESCR': '.. _california_housing_dataset:\n\nCalifornia Housing dataset\n-----
-----\n\n**Data Set Characteristics:**\n\n:Number of Instances: 206
40\n\n:Number of Attributes: 8 numeric, predictive attributes and the target\n\n:
Attribute Information:\n    - MedInc            median income in block group\n    - H
ouseAge      median house age in block group\n    - AveRooms      average number
of rooms per household\n    - AveBedrms      average number of bedrooms per househ
old\n    - Population      block group population\n    - AveOccup      average numb
er of household members\n    - Latitude      block group latitude\n    - Longitud
e      block group longitude\n\n:Missing Attribute Values: None\n\nThis dataset wa
s obtained from the StatLib repository.\nhttps://www.dcc.fc.up.pt/~ltorgo/Regress
ion/cal_housing.html\n\nThe target variable is the median house value for Califor
nia districts,\nexpressed in hundreds of thousands of dollars ($100,000).\n\nThis
dataset was derived from the 1990 U.S. census, using one row per census\nblock gr
oup. A block group is the smallest geographical unit for which the U.S.\nCensus B
ureau publishes sample data (a block group typically has a population\nof 600 to
3,000 people).\n\nA household is a group of people residing within a home. Since
the average\nnumber of rooms and bedrooms in this dataset are provided per househ
old, these\ncolumns may take surprisingly large values for block groups with few
households\nand many empty houses, such as vacation resorts.\n\nIt can be downloa
ded/loaded using the\n:func:`sklearn.datasets.fetch_california_housing` functio
n.\n\n.. rubric:: References\n\n- Pace, R. Kelley and Ronald Barry, Sparse Spatia
l Autoregressions,\n    Statistics and Probability Letters, 33:291-297, 1997.\n'}
```

In [7]:

```
# train/test
x=df.drop(columns='MedHouseVal')
y=df['MedHouseVal']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

In [8]:

```
#model
model=LinearRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
```

In [10]:

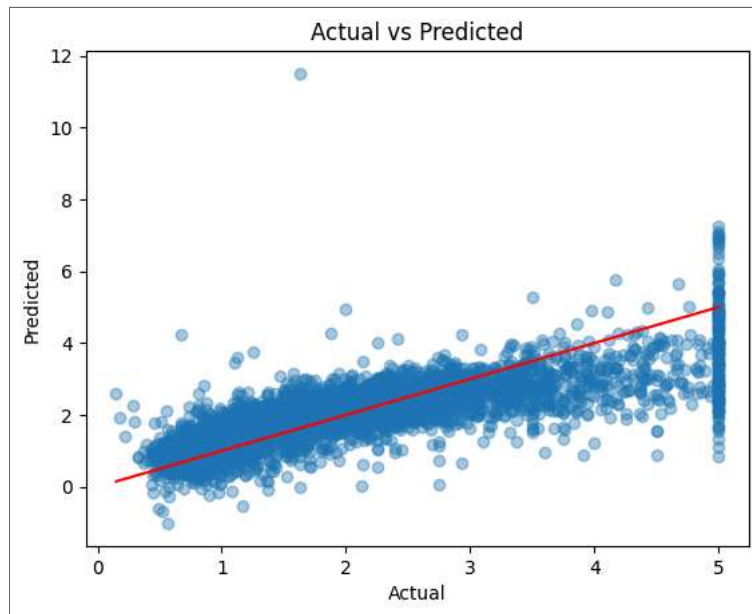
```
#metrics
y_pred = model.predict(x_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.3f}, RMSE: {rmse:.3f}, R2: {r2:.3f}")
```

MAE: 0.533, RMSE: 0.746, R2: 0.576

In [11]:

```
# plot
plt.scatter(y_test, y_pred, alpha=0.4)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.show()
```



In []: